

Final Project **Weather App**

A modern, reactive weather and air quality monitoring application
built natively for Android.

Presented by: Liran Cordova, Or Rodrigez, Or Zuka

Project Summary & Purpose



Core Summary

An Android application providing real-time weather data, detailed 5-day forecasts, and air quality (pollution) information powered by the OpenWeatherMap API.



Primary Purpose

Offer users a clean interface to monitor their local environment and manage favorite global locations for quick access.



Target Audience

Designed for general mobile users, frequent travelers, and health-conscious individuals who need reliable monitoring of varying air quality levels.

App Screen Flow (1/2)



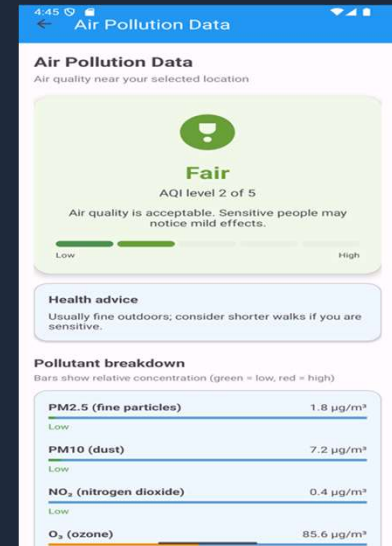
Home Fragment

Displays current temperature, core weather conditions, and main quick actions.



Forecast Fragment

Shows a detailed, scrollable list of the next 5 days predicted weather.



Pollution Fragment

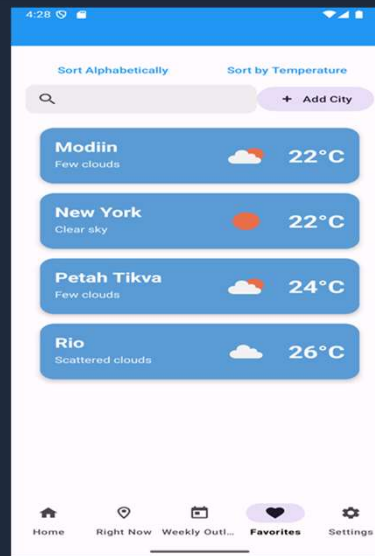
Detailed breakdown of the AQI (Air Quality Index) and specific pollutant metrics.

App Screen Flow (2/2)



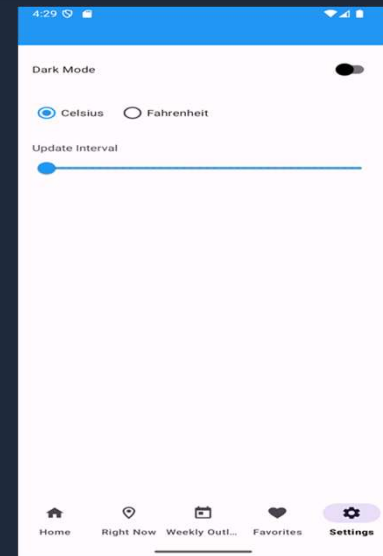
📅 Forecast Fragment

A dynamic weather forecast list



★ Favorites Fragment

A list of saved cities stored in the local Room database for quick access.



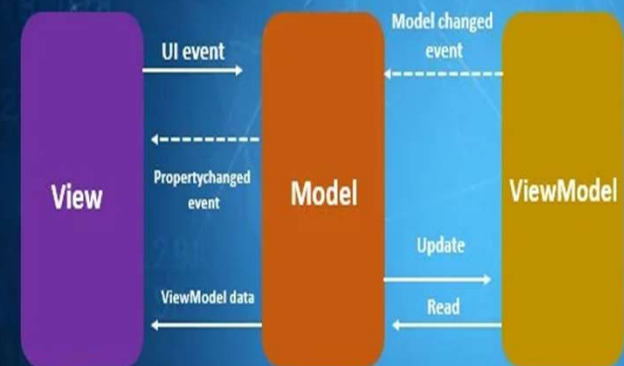
⚙️ Settings Fragment

Options for unit toggling (Celsius/Fahrenheit) and notification preferences.

Project Architecture

-  **MVVM Pattern** Utilizes Model-View-ViewModel to separate UI components from business logic and data state.
-  **Dependency Injection** Implements Dagger-Hilt for modular, scalable, and easily testable code architecture.
-  **Remote Data Flow** Retrofit handles fetching structured JSON data from the OpenWeatherMap REST API.
-  **Local Data Flow** Room Persistence Library securely caches data for offline access and manages Favorites.

MVVM Architecture



External Libraries



Networking (Retrofit & Gson) Handles asynchronous API communication and JSON serialization robustly.



Database (Room) Provides an abstraction layer over SQLite for smooth local data storage.



Injection (Dagger-Hilt) Manages lifecycle-aware dependency injection automatically across the app.



Image Loading (Glide) Efficiently fetches and caches high-quality weather icons from the web.



Background Tasks (WorkManager) Ensures scheduled weather updates execute reliably even if the app is closed.